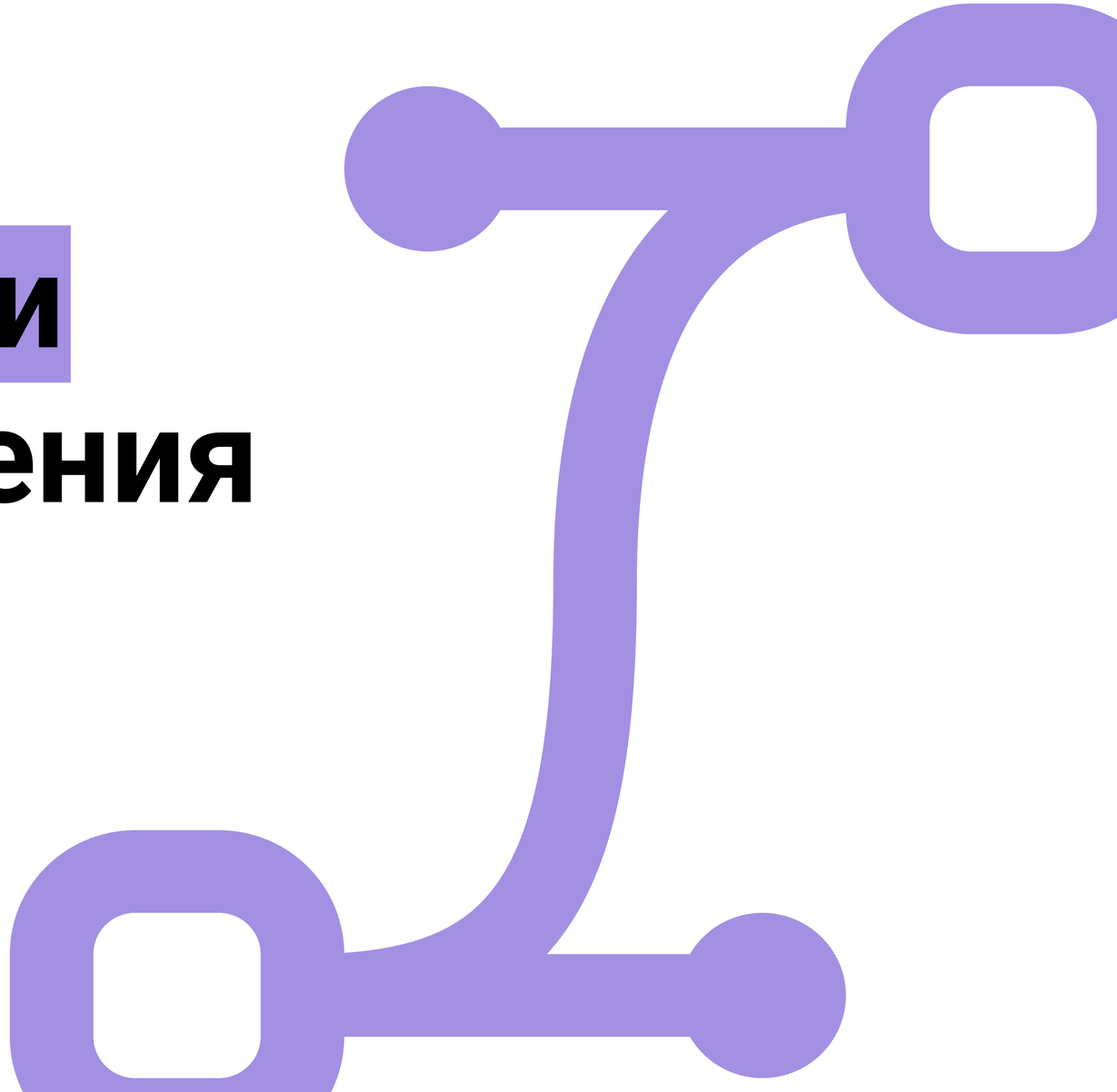
2026

Платформы и среды выполнения ЯП

NEXIGN x ИТМО

Раздел 02

Ландшафт памяти и модели исполнения



Выравнивание

```
1  class Slow {
2      char a;
3      int b;
4      char c;
5  };
6
7  // ??? байт
```

```
1  class Fast {
2      int b;
3      char a;
4      char c;
5  };
6
7  // ??? байт
```

Выравнивание

```
class Slow {
    char a;
    int b;
    char c;
};

// 12 байт
```

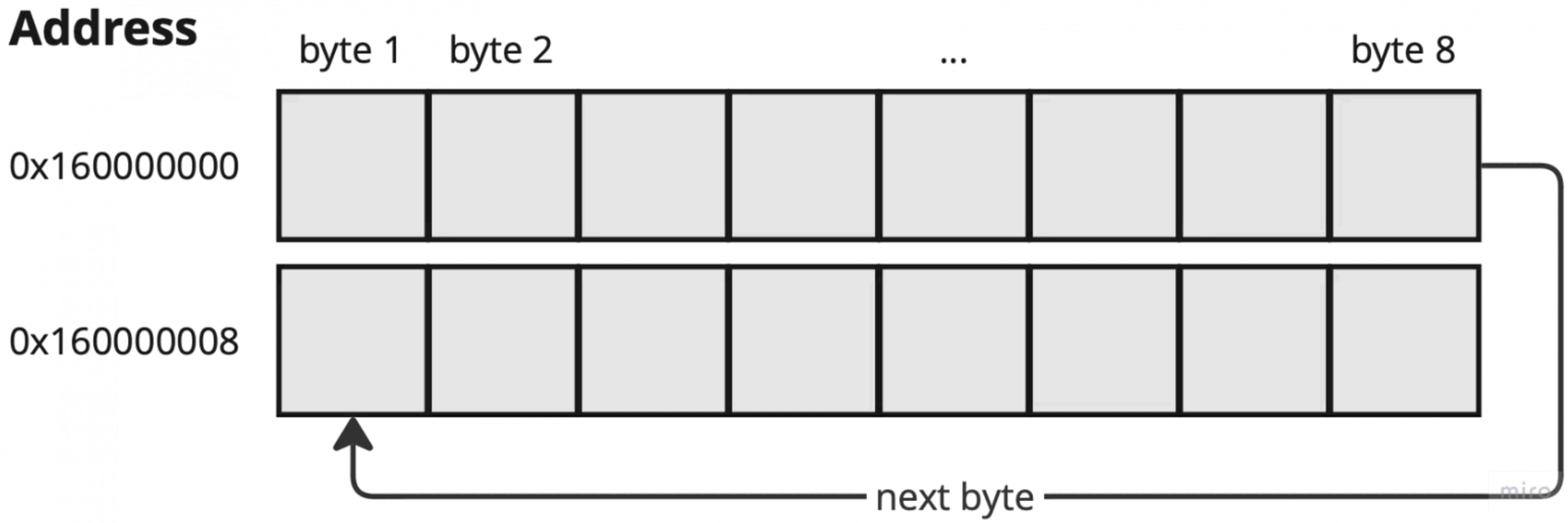
```
class Fast {
    int b;
    char a;
    char c;
};

// 8 байт
```

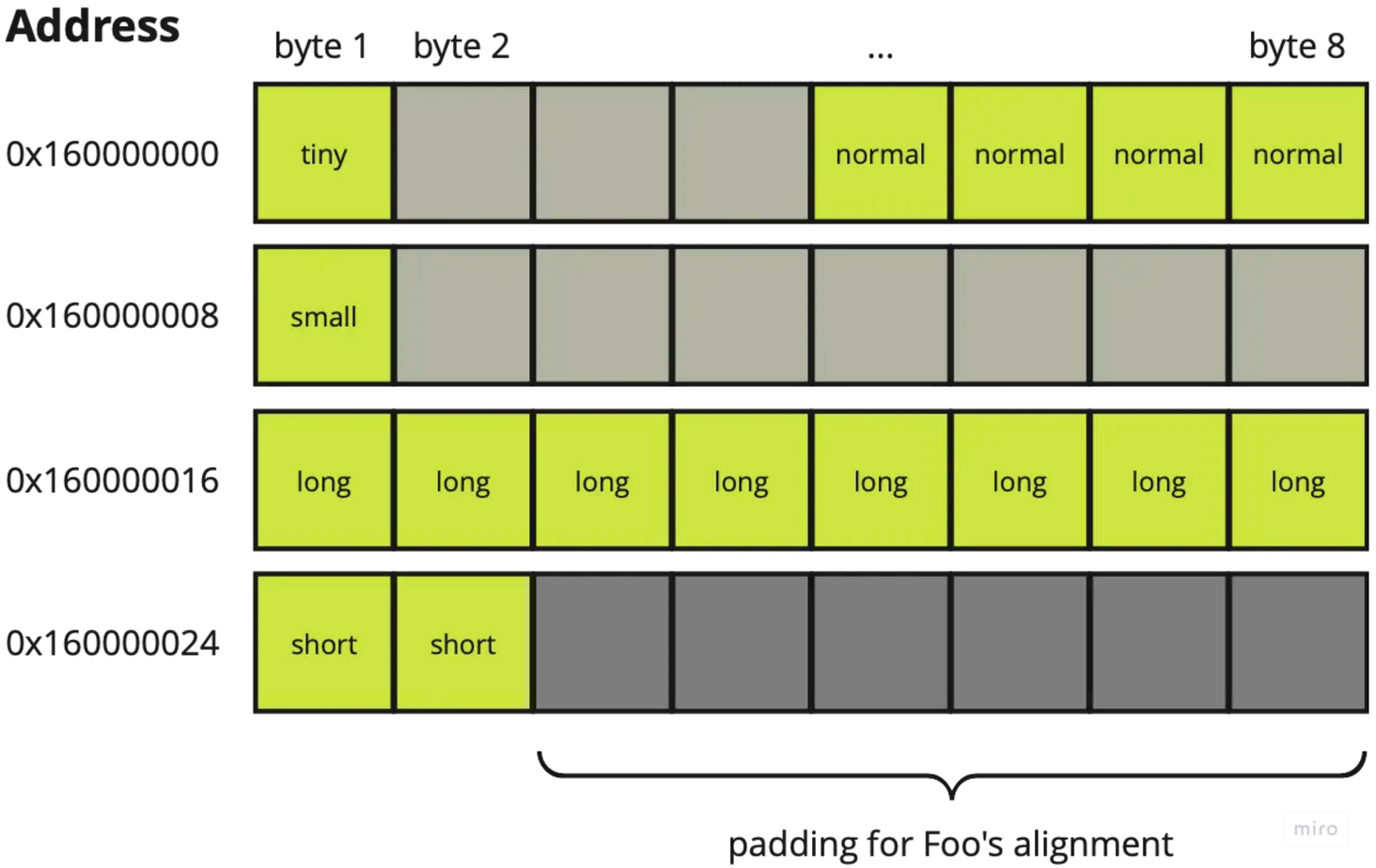
Выравнивание

```
1  struct A { char x; char y; char z; };
2  struct B { char x; int y; char z; };
3  struct C { int x; char y; char z; };
4  struct D { double x; int y; char z; };
5
6  // sizeof() == ???
```

Выравнивание



Выравнивание



Причины выравнивания

- Процессор читает память словами, а не по одному байту.
- Выровненные данные считываются за одно обращение к памяти.
- Невыровненные данные требуют двух обращений и сдвига.
- На некоторых архитектурах (старые ARM, MIPS) невыровненный доступ вызывает аппаратное исключение.
- На x86-64 и современных ARM невыровненный доступ работает, но с штрафом производительности.

Масштаб проблемы

- Один объект: разница 4 байта.
- Миллион объектов: разница 4 МБ.
- Миллиарды объектов...

- Больше объектов в памяти — больше кэш-линий — медленнее обход.
- Следствие: колоночные базы данных (ClickHouse, Apache Arrow) хранят данные в плотных массивах для минимизации заполнения.

Что с этим делать?

- Переупорядочивание полей: крупные поля первыми, мелкие последними. Компилятор не делает этого автоматически.
- alignas(64) в C++11: явное выравнивание по границе кэш-линии. Надёжный и переносимый инструмент.
- #pragma pack / __attribute__((packed)): отключение заполнения (для сериализации, сетевых протоколов). Но! Невыровненный доступ медленнее.
- std::hardware_destructive_interference_size (C++17): подгон выравнивания под размер кэш-линии на целевой платформе.

Выравнивание в управляемых средах

- JVM: порядок полей может изменяться сборщиком мусора. Заголовок объекта — 8 или 16 байт.
- CLR: `[StructLayout(LayoutKind.Explicit)]` для явного задания смещений. По умолчанию CLR оптимизирует раскладку.
- При межъязыковом взаимодействии (JNI, P/Invoke) разработчик должен учитывать выравнивание явно.

Кэш

Время доступа к памяти:

- Кэш L1: ~1 нс
- Кэш L2: ~4 нс
- Кэш L3: ~10 нс
- Оперативная память: ~100 нс

Кэш-промах стоит десятки или сотни тактов процессора...

Кэш-линия

- При кэш-промахе из памяти загружается блок фиксированного размера (кэш-линия).
- Обращение к одному байту загружает в кэш всю кэш-линию.
- Аппаратная основа пространственной локальности: если программа обратилась к элементу i, следующий элемент i+1 уже в кэше.

Размер кэш-линии зависит от целевой платформы (32, 64, 128...)

Обход матрицы

```
1   for (i = 0; i < N; i++)
2       for (j = 0; j < N; j++)
3           sum += matrix[i][j];
```

```
1   for (j = 0; j < N; j++)
2       for (i = 0; i < N; i++)
3           sum += matrix[i][j];
```

Обход матрицы

```
1   for (i = 0; i < N; i++)
2       for (j = 0; j < N; j++)
3           sum += matrix[i][j];
```

```
1   for (j = 0; j < N; j++)
2       for (i = 0; i < N; i++)
3           sum += matrix[i][j];
```

Разница: ~25 раз
(на матрице 4096x4096)

Почему это происходит

Схема кэш-линий для матрицы:

- Строчный обход: элементы строки лежат последовательно. Одна кэш-линия содержит 16 элементов `int`. Каждый 16-й элемент — промах, остальные 15 — попадания.
- Столбцовый обход: элементы столбца разделены шагом `N * 4` байт. Каждый элемент попадает в новую кэш-линию. Каждый доступ — промах.

Менее очевидно

```
// Массив структур (AoS)
struct Particle { float x, y, z, vx, vy, vz; };
Particle particles[1000000];

// Массив полей (SoA)
float x[1000000], y[1000000], z[1000000];
float vx[1000000], vy[1000000], vz[1000000];

// AoS: каждая кэш-линия содержит одну частицу целиком (24 байта из 64 используются при доступе только к координатам).
// SoA: каждая кэш-линия содержит 16 значений `x` подряд. Все 64 байта используются.
// Разница в количестве кэш-промахов: двукратная и более.
```

Реальные примеры SoA

- Игровые движки: архитектура Entity-Component-System (ECS) хранит компоненты в плотных массивах по типу.
- Колоночные базы данных: ClickHouse, Apache Parquet, Apache Arrow хранят данные по колонкам.
- Финансовые системы и системы машинного обучения: SoA-раскладка для эффективного использования SIMD-инструкций.

Ложное разделение

- Ложное разделение (false sharing) — проблема производительности в многопоточном коде.
- Несколько потоков модифицируют разные переменные, находящиеся в одной кэш-линии.
- Протокол когерентности кэшей (MESI) заставляет кэш-линию мигрировать между ядрами.
- Данные логически не разделяются, но аппаратно связаны через кэш-линию.

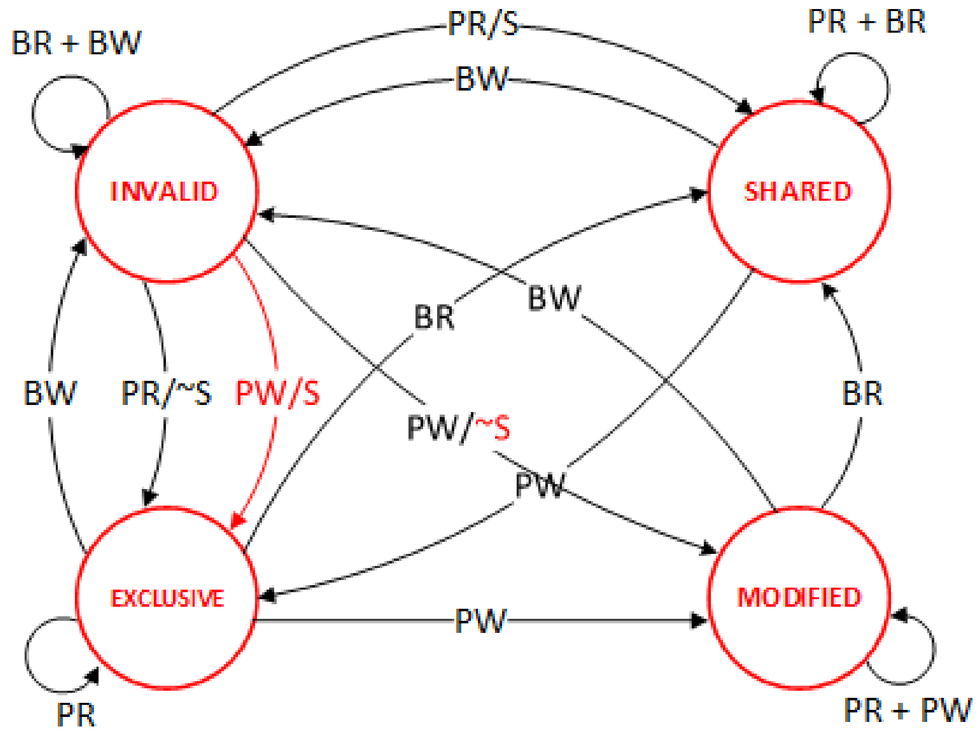
Классический пример

```
int counters[NUM_THREADS];

void worker(int thread_id) {
    for (int i = 0; i < ITERATIONS; i++)
        counters[thread_id]++;
}

// Потоки инвалидируют кэш друг друга
// Производительность падает в 10–100 раз
```

Схема протокола MESI

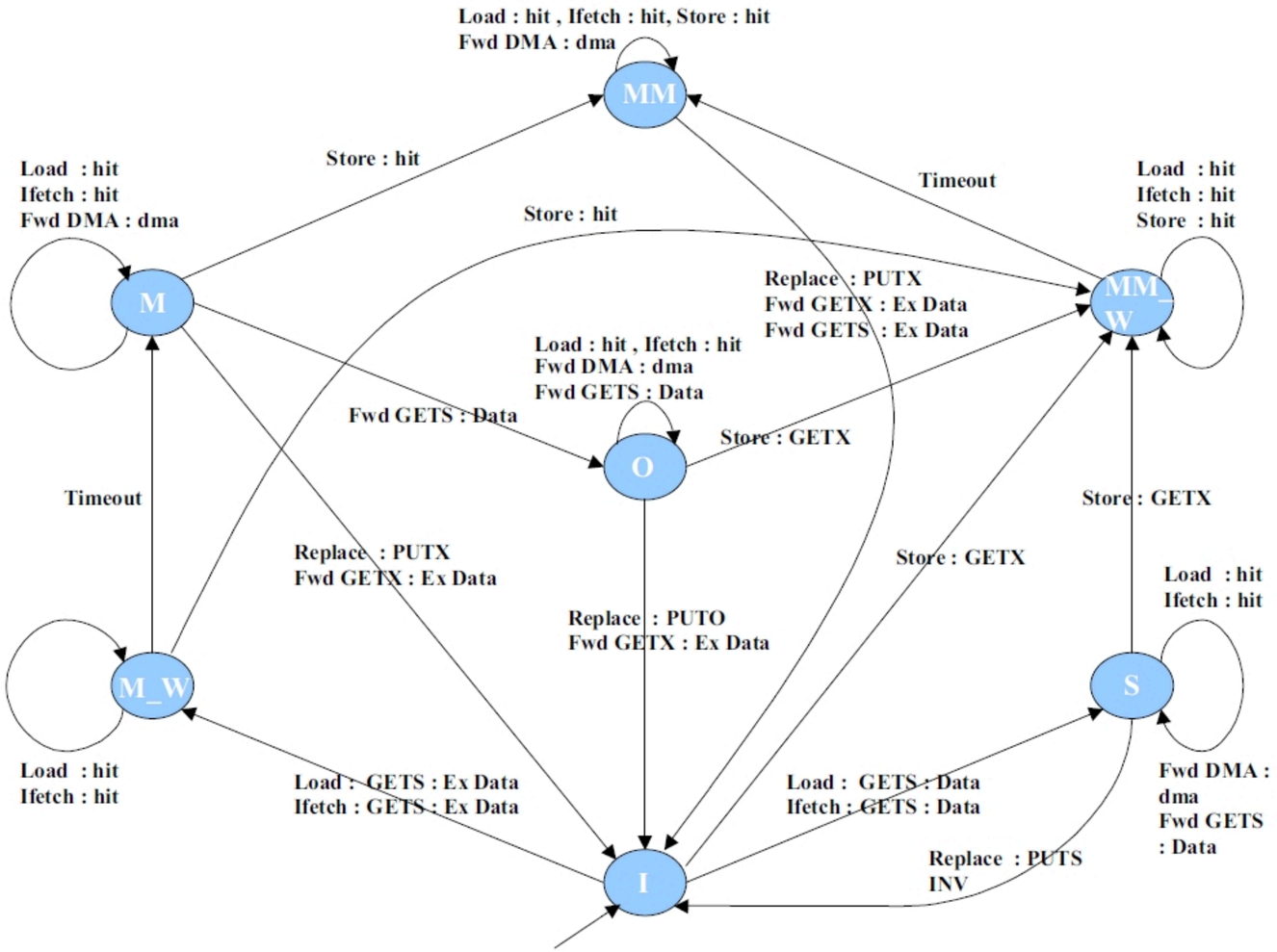


PR	= processor read	BR	= observed bus read
PW	= processor write	BW	= observed bus read
S/~S	= shared/NOT shared		

Схема протокола MOESI







LMAX Disruptor

- Высокопроизводительная система обмена сообщениями для финансовой торговли.
- Обрабатывает миллионы транзакций в секунду.
- Разработчики явно используют заполнение для разделения записей кольцевого буфера по кэш-линиям.
- Без заполнения производительность падает на порядок.
- Документация проекта содержит подробное объяснение и бенчмарки.

Идеи Disruptor в Go и Rust

```go
type PaddedCounter struct {
    value int64
    _     [56]byte // дополнение до 64 байт
}
```

```rust
#[repr(align(64))]
struct PaddedAtomic {
    value: AtomicU64,
}
```

Итого

1. Заполнение (padding):
   - C++: `alignas(64) int counter;` или ручное заполнение.
   - Java: объекты-обёртки с массивами для заполнения.
   - Go: `_ [64]byte` или `_ [128]byte` в структуре.
   - Rust: `#[repr(align(64))]` на структуре.
2. Проверка размера кэш-линии на целевой платформе (не полагаться на фиксированное значение 64).
3. `std::hardware_destructive_interference_size` (C++17): использовать осторожно, проверять на целевой платформе.
4. Перепроектирование данных: группировка данных по потокам.
5. Профилирование: `perf c2c` (Linux) для обнаружения ложного разделения.